

ActMe / 端侧 Agent 创意设计

从工具集合到端侧执行闭环

围绕 ADB 截图配对、端侧 OCR、GUI Sub-Agent 与主 Agent 协作链路，ActMe 探索让移动端 Agent 不止会回答，还能在授权范围内观察、计划、执行和复盘。

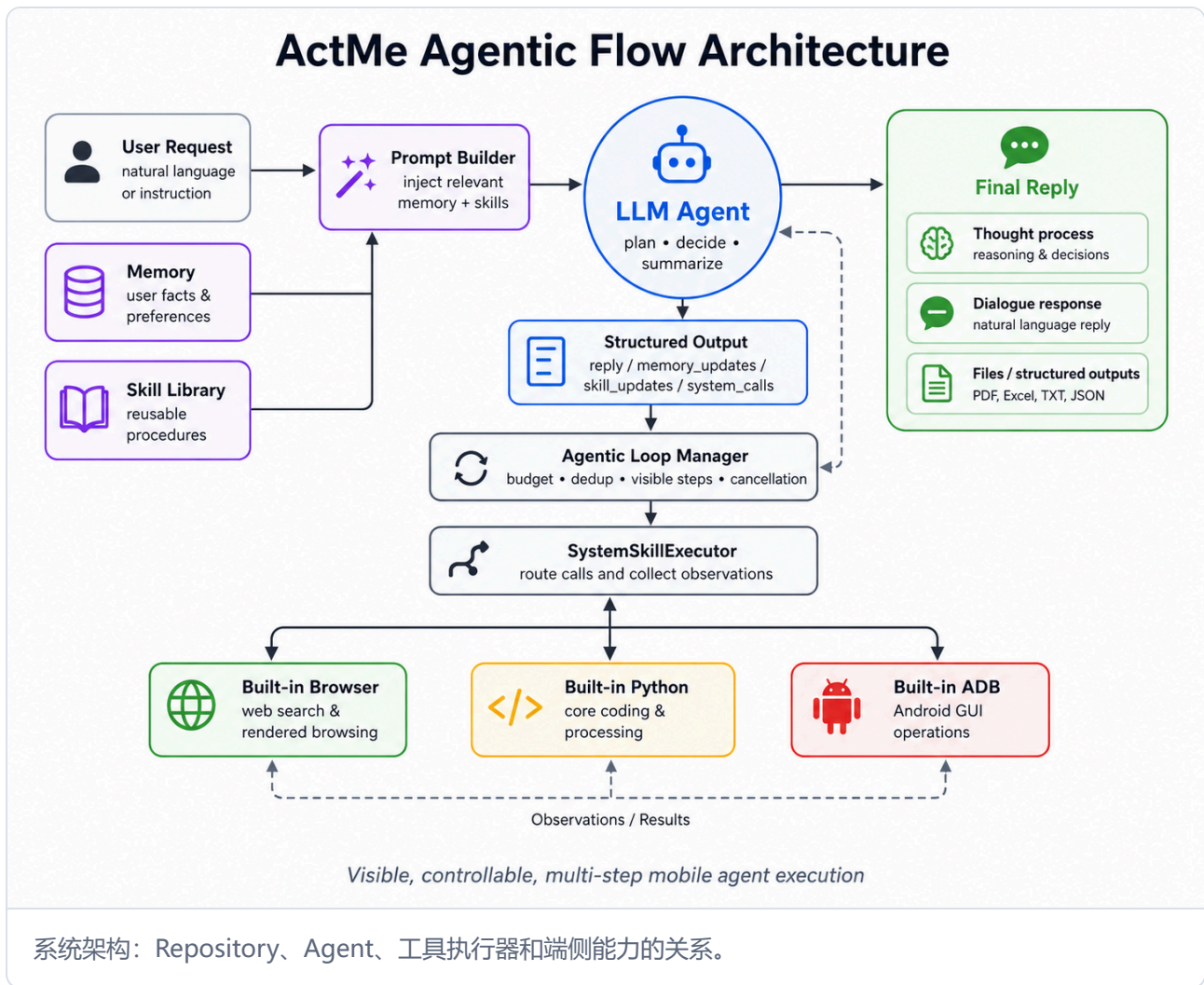
ADB + OCR + GUI-Owl

Agentic Loop

ActMe 的核心探索是把云端语言规划和端侧感知执行拆开：云端主 Agent 负责意图、计划和纠错，端侧 App 负责无线 ADB 连接、截图 OCR、视觉子 Agent、坐标映射和真实操作。这让手机本身成为 Agentic Loop 的执行现场。



ActMe 功能总览：聊天 Agent、记忆、日程、本地模型与工具执行入口。



[本地 ADB + OCR + GUI Sub-Agent 链路](#)

[内置能力总览](#)

[Agentic Loop 设计](#)

[内置 ADB](#)

[内置浏览器](#)

[内置 Python](#)

[技能记忆设计](#)

LOCAL_ADB_OCR_GUI_SUBAGENT.MD

本地 ADB + OCR + GUI Sub-Agent 链路

这条链路是 ActMe 在端侧 Agent 执行能力上的一次探索：让云端主 Agent 负责意图理解、任务拆解和失败后的修正，让端侧 App 负责设备连接、截图感知、视觉判断和真实 UI 操作。它不是把手机变成一个简单的 `adb shell` 终端，而是把无线 ADB、端侧 OCR、端侧视觉模型和 Agentic Loop 串成一个可见、可中止、可复盘的执行闭环。

设计目标

- 降低无线 ADB 配对门槛：用户只需要在系统无线调试页面截图，App 监听图库新增图片并尝试自动识别配对信息。
- 把 OCR 留在端侧：GLM-OCR-MNN 只做图片文字转写，规则代码再从文本中解析设备名、IP、连接端口、配对端口和配对码。
- 区分规划与执行：云端主 Agent 不直接看屏幕，只写 `gui_agent.plan`、`target_text` 和必要的 `guidance`；本地 GUI 子 Agent 看截图并执行当前一步。
- 避免手写 ADB 点控链：第三方 App 操作必须走 `gui_agent`，`adb_shell` 主要用于诊断、只读观察、日志、包名、截图和用户明确要求的低层命令。
- 保留端侧安全边界：执行过程显示 GUI 悬浮提示，步骤进入聊天工具气泡，失败结果返回主 Agent 继续修正，而不是静默乱点。

总体链路

用户提出手机 GUI 任务

- > 云端主 Agent 判断需要 `gui_agent`
- > 写入 `command / plan / target_text / guidance`
- > `SystemSkillExecutor` 调用 `GuiSubAgent`
- > `GuiSubAgent` 检查无线 ADB 是否可用
 - > 不可用：启动截图监听，打开开发者设置，引导用户截图无线调试配对页
 - > 可用：截图、调用端侧 `GUI-Owl`、解析动作、执行 ADB 操作
- > 返回 `[GUI_AGENT_RESULT] / [GUI_AGENT_ACTION_ERROR] / [GUI_AGENT_NEEDS_ADB]`
- > 主 Agent 根据观察结果继续规划或给出最终回复

这条链路的关键不在于某一次点击成功，而在于把“看见手机屏幕 - 决定下一步 - 执行动作 - 再观察”的循环放进移动端 App 内部。ActMe 因此开始具备端侧 Agent 的基础形态：本地有感知、本地有执行、本地有可见状态，云端只承担语言层面的计划和纠错。

无线 ADB 截图配对

相关代码：

```
app/src/main/java/com/actme/app/data/agent/AdbPairingScreenshotService.kt
app/src/main/java/com/actme/app/data/agent/AdbPairingScreenshotWatcher.kt
app/src/main/java/com/actme/app/image/LocalImageImportManager.kt
app/src/main/java/com/actme/app/data/agent/AdbSkillEngine.kt
app/src/main/java/com/actme/app/ui/settings/SettingsScreen.kt
```

设置页的“内置 ADB”不再只弹出手动配对窗口，而是启动图库新增截图监听，并尝试打开系统开发者设置。用户在无线调试配对详情页截图后，`AdbPairingScreenshotWatcher` 会：

- 记录启动时最新图片作为 baseline。
- 通过 `MediaStore` observer 和轮询检测新的图片。
- 把新图复制到 App 私有目录，并按 OCR 模型输入限制缩放。
- 调用 `LocalImageImportManager.extractText()` 使用 GLM-OCR-MNN 做端侧 OCR。
- 用确定性规则解析 ADB 连接信息。
- 调用 `AdbSkillEngine.pair()` 完成配对，再调用 `testConnection()` 测试并保存连接端口。

OCR 模型不负责“理解 ADB 配对流程”。它只输出可见文字；设备名称、地址、端口和配对码由规则解析。这一点很重要：端侧视觉模型负责感知，确定性代码负责高风险连接参数提取，减少模型幻觉进入 ADB 连接层的机会。

GUI Sub-Agent 执行器

相关代码：

```
app/src/main/java/com/actme/app/data/agent/GuiSubAgent.kt
app/src/main/java/com/actme/app/ui/GuiAgentOverlayService.kt
app/src/main/java/com/actme/app/data/agent/AdbKeyboardInput.kt
app/src/main/java/com/actme/app/data/agent/SystemSkillExecutor.kt
```

`GuiSubAgent` 是本地视觉执行器。它接收主 Agent 的任务、计划、固定输入文本和纠错指导，然后在端侧循环执行：

检查 ADB

- > 必要时预启动目标 App
- > ADB 截图到本地文件
- > 构造带截图的 GUI-Owl prompt
- > 模型返回单个 JSON 动作
- > 归一化坐标映射到真实像素
- > ADB 执行 tap / swipe / keyevent / 输入
- > 记录结果并进入下一步

当前允许的动作包括 `open_app`、`click`、`long_press`、`swipe`、`type`、`system_button`、`wait`、`answer` 和 `terminate`。每一轮只允许一个动作，避免模型一次性输出一串不可验证的操作。

输入文本是单独的安全通道。主 Agent 必须把目的地、搜索词、消息内容等写入 `target_text`；本地 GUI 子 Agent 在执行 `type` 时会强制使用 `target_text`，即使视觉模型输出了不同文本也会覆盖。对于中文或复杂文本，`AdbKeyboardInput` 会优先尝试普通 `input text`，失败后安装并切换随包携带的 ADBKeyboard，通过 base64 broadcast 输入，再恢复原输入法。

主 Agent 协议变化

`SystemCall` 新增字段：

```
{
  "type": "gui_agent",
  "command": "Open Amap and navigate to destination",
  "plan": "1. Open package com.autonavi.minimap from launcher.\n2. Tap the search box.\n3. Type target_text exactly.",
  "target_text": "华东师范大学普陀校区",
  "guidance": "",
  "timeout_ms": 120000
}
```

`ActMeAgent` 的系统提示中明确了分工：

- `adb_shell` 只做诊断、只读观察、日志、包列表、设置查询、截图，或者用户明确要求的低层 ADB 命令。
- 任何操作第三方 App UI 的任务都必须走 `gui_agent`。
- 主 Agent 是 planner，不能假装看见屏幕；本地 GUI 子 Agent 是 visual executor，负责根据截图执行当前一步。

- GUI 子 Agent 失败后，主 Agent 应阅读返回的 `[GUI_AGENT_ACTION_ERROR]` 或 `[GUI_AGENT_ERROR]`，更新 plan/guidance 后再次调用，而不是直接放弃。

为了提高启动成功率，主 Agent 在检测到“打开、进入、点击、导航、搜索、发消息、设置、地图、微信、支付宝”等 GUI 意图时，会注入常见国产 App 包名参考，要求 `gui_agent.plan` 的第一步写清楚目标包名。端侧执行器也会从任务和计划中推断包名并预启动目标 App。

可见性与调试

这条链路强调可观察，而不是把自动化藏起来：

- ADB 配对截图监听以前台服务通知展示。
- GUI 执行时右上角显示 `ActMe GUI running` / `ActMe GUI step N` 悬浮提示。
- `SystemSkillExecutor` 在聊天气泡里显示 `Run GUI Agent`。
- `GuiSubAgent` 记录模型输入、模型输出、截图尺寸、坐标缩放、执行结果和错误。
- 坐标既支持 0-1000 归一化，也支持绝对像素；执行前统一映射到真实截图尺寸并写日志。

这些日志让端侧 Agent 的失败可复盘：到底是主 Agent 计划错、视觉模型看错、坐标映射错、ADB 操作失败，还是目标 App 状态变化导致流程卡住。

端侧 Agent 探索意义

当前链路的探索性体现在四个层面：

- 感知本地化：截图 OCR 和 GUI 视觉判断发生在手机端，用户屏幕不必作为原始图像交给云端主 Agent。
- 执行本地化：点击、输入、返回、截图和应用启动通过本地无线 ADB 完成，形成真正的端侧执行通道。
- 规划分层：云端模型做语言规划和失败修正，端侧模型做视觉一步执行，降低单个模型同时承担“理解需求 + 看图 + 控制设备”的复杂度。
- 安全收敛：高权限 ADB 被约束在工具层，第三方 App GUI 操作被收敛到 `gui_agent`，输入文本被 `target_text` 固定，失败返回结构化观察供下一轮修正。

因此，这不是传统脚本自动化，也不是纯云端 mobile-use。它更接近一个移动端个人 Agent 的早期原型：在本机拥有感知、连接、执行、提示和日志，能够围绕真实 App 状态做多步闭环。

当前边界

- 无线 ADB 仍依赖用户主动开启无线调试、授予图片读取权限，并在配对页截图。
- OCR 对系统页面布局、截图清晰度和端口文本可见性敏感；规则解析会拒绝缺失字段或配对端口等于连接端口的异常结果。
- GUI-Owl 每轮只执行一步，复杂任务需要多轮截图和模型调用，速度不是脚本级别。
- 坐标点击仍可能受弹窗、动画、输入法、系统权限页遮挡影响。
- 高风险 ADB 命令仍应保留用户确认，不应让 GUI Agent 绕过用户授权边界。

这些边界是当前设计刻意保留的约束。ActMe 现阶段追求的是可控的端侧 Agent 链路，而不是无人值守、不可解释的全自动手机控制。

BUILTIN_CAPABILITIES.MD

ActMe 内置能力总览：三驾马车

ActMe 的 Agent 能力不是只依赖大模型本身，而是由三类 App 内置执行能力共同支撑：

- 内置浏览器：让 Agent 能联网搜索、打开网页、读取渲染后的页面内容。
- 内置 Python：让 Agent 能做确定性计算、数据处理、Excel 读写、通用文件生成和脚本复用。
- 内置 ADB：让 Agent 在用户授权后具备观察和操作本机 Android 环境的能力。

这三类能力构成 ActMe 的执行层。大模型负责规划、判断、生成工具调用和总结结果；App 负责真实执行、展示步骤、返回观察结果，并允许用户中止。

设计目标

ActMe 的目标是把移动端 Agent 从“会聊天”推进到“能执行”：

- 遇到最新信息、事实核验、网页资料时，调用内置浏览器。
- 遇到计算、解析、表格、Excel、文件生成时，调用内置 Python。
- 遇到本机 App 状态、系统页面、UI 自动化、日志/截图/输入操作时，调用内置 ADB。

三者组合后，Agent 可以完成更长链路任务，例如：

- 搜索多个网页，浏览权威来源，用 Python 清洗和汇总数据，生成 Excel 或 PDF。
- 浏览官网和公告，提取关键文本，用 Python 比对表格，再给出结论。

- 使用 ADB 读取当前手机界面状态，执行输入/点击，再观察结果继续操作。

System Calls

Agent 通过 `system_calls` 请求 App 执行工具。典型格式：

```
{
  "system_calls": [
    {
      "type": "web_search",
      "query": "中国银行 积存金 价格"
    },
    {
      "type": "browse_url",
      "url": "https://www.boc.cn/fimarkets/"
    },
    {
      "type": "python_exec",
      "code": "emit({'answer': 2 + 2})",
      "input": "",
      "timeout_ms": 3000
    },
    {
      "type": "html_to_pdf",
      "url": "reports/report.html",
      "output_files": ["reports/report.pdf"]
    },
    {
      "type": "adb_shell",
      "command": "dumpsys window | head -50",
      "timeout_ms": 15000
    }
  ]
}
```

App 执行后会把结果写回多步执行循环，Agent 可以观察结果后继续调用更多工具，直到信息足够或工具预算耗尽。

用户可见性

三驾马车不是后台黑盒。ActMe 会在聊天中展示工具执行步骤：

- 正在搜索什么。

- 正在打开哪个网页。
- 正在运行 Python。
- 正在执行 ADB 命令。
- 哪一步成功、失败或被跳过。

用户可以在执行过程中中止当前任务。

能力边界

三驾马车提升的是执行能力，不等于完全自动自治：

- 浏览器读取网页可能受登录、反爬、地区、动态渲染和网站结构影响。
- Python 沙箱允许导入标准库和已安装包；Python 层不额外拦截文件读写删改，实际权限交给 Android App 沙箱和系统权限决定；进程、native code、包安装和系统 shell 等危险能力仍受限。
- 报告类文件优先通过 `write_report` 生成 Markdown 和 HTML；需要 PDF 时调用 `html_to_pdf`，由 Android WebView 渲染 HTML 并写出 PDF。
- ADB 需要用户手动授权无线调试和悬浮窗权限，且连接端口可能随系统状态变化。
- 高风险操作需要谨慎，尤其是 ADB 的删除、卸载、清数据、权限修改等命令。

文档入口

- [内置浏览器](BUILTIN_BROWSER.md)
- [内置 Python](BUILTIN_PYTHON.md)
- [内置 ADB](BUILTIN_ADB.md)

AGENTIC_LOOP.MD

Agentic Loop 设计

ActMe 的 Agentic Loop 是让 Agent 在一次用户请求中多轮调用工具、观察结果、继续决策的执行框架。它的目标是支持复杂任务，而不是只做单轮问答。

目标

Agentic Loop 解决的问题：

- 用户的问题可能需要多步完成。
- 联网结果需要继续打开网页验证。
- 网页文本需要 Python 清洗和计算。
- ADB 操作需要观察当前 UI，再执行点击/输入，再继续观察。
- 工具调用失败时需要降级或换路径。

因此 ActMe 允许 Agent 在一次回复过程中反复执行：

```
思考 -> system_calls -> App 执行工具 -> 观察结果 -> 继续思考 -> 更多工具或最终回复
```

核心流程

主要代码位置：

```
app/src/main/java/com/actme/app/data/repo/ActMeRepository.kt
app/src/main/java/com/actme/app/data/agent/ActMeAgent.kt
app/src/main/java/com/actme/app/data/agent/SystemSkillExecutor.kt
```

职责划分：

- `ActMeAgent`：构造提示词、调用 LLM、解析 JSON 输出和 `system_calls`。
- `ActMeRepository`：驱动多步循环、管理工具预算、写入聊天消息和步骤展示。
- `SystemSkillExecutor`：执行具体工具，包括浏览器、Python、ADB、时间等。

Agent 输出协议

Agent 每轮输出 JSON：

```
{
  "reply": "",
  "memory_updates": [],
  "schedule_updates": [],
  "skill_updates": [],
  "system_calls": [
    {
      "type": "web_search",
      "query": "搜索内容"
    }
  ]
}
```

如果 `system_calls` 非空, `reply` 可以为空。App 会先执行工具, 再把结果作为下一轮输入交给模型。

多步执行

一次用户消息可能触发多个 pass:

- 第一轮模型判断需要工具。
- App 执行工具。
- App 把工具结果拼入 continuation input。
- 模型基于工具结果继续输出:
 - 继续调用更多工具。
 - 或生成最终回复。
- 达到工具预算或信息足够后停止。

工具结果会用分隔符拼接, 避免不同工具输出混淆。

工具预算

为了防止无限循环, ActMe 对工具调用做预算控制:

- 总工具调用数。
- 搜索调用数。
- 浏览调用数。
- 多步 pass 上限。

当预算耗尽时，App 会暂停工具循环，让 Agent 基于已有结果给出当前最好回答。若还有后续工具请求，用户可以发送“继续”让任务继续执行。

从 1.3.0 开始，循环预算提高到更适合报告、网页调研和多文件生成的规模，同时仍保留去重和上限保护。对于“模型空回复且没有任何输出文件”的 pass，Repository 会自动触发恢复提示，让 Agent 继续生成最终回复或继续完成用户要求的结果，而不是把空消息交给用户。

去重策略

ActMe 会避免重复执行明显相同的工具调用：

- 相同 `web_search.query` 不重复搜索。
- 相同 `browse_url.url` 不重复浏览。
- 相同 Python 代码和输入不重复执行。
- 相同 ADB 命令不重复执行，除非 Agent 给出新命令或用户继续要求。

去重的目标是减少死循环和无效消耗。

可见执行步骤

Agentic Loop 的每一步都会进入聊天 UI 的工具执行气泡：

- 工具开始。
- 工具完成。
- 工具失败。
- 工具被跳过。
- 工具循环暂停。

这让用户能看到 Agent 正在做什么，也方便调试。

可中止

用户可以在执行过程中中止当前任务。中止后：

- 当前 coroutine 会取消。
- 工具执行气泡显示已中止。
- 不再继续调用后续工具。

这对联网、浏览、Python 和 ADB 都很重要，因为这些操作可能耗时或产生副作用。

文件输出收集

从 1.2.0 开始，Agentic Loop 把生成文件作为一等输出处理。任意 pass 的任意工具都可能产生文件，尤其是 `python_exec`。

处理策略：

- `SystemSkillExecutor` 合并模型声明的 `output_files` / `generated_files` / `expected_outputs` / `files`。
- Python 执行器会自动检测 `agent_workspace` 中新增或修改的文件。
- `ActMeRepository` 在每一轮工具执行结束后立即收集文件引用。
- 最终回复前再次扫描累计工具结果和最终 reply。
- 使用有序集合去重，避免重名路径重复显示。
- Chat UI 会从消息正文和隐藏的工具结果中识别文件，并显示打开按钮。

这意味着即使 Agent 没有在最终自然语言回复中提到某个文件，只要工具真实生成并返回到 observation，最终聊天气泡也应显示文件入口。

报告生成是 1.3.0 的重点路径。Agent 应优先通过 `write_report(markdown_text, "reports/name", title="...")` 生成 Markdown 和 HTML；需要 PDF 时继续调用 `html_to_pdf`，由 Android WebView 把 HTML 渲染为 PDF。任意 pass 中产生的 `.md`、`.html`、`.pdf`、Excel、CSV、图片或文本文件都会被累计收集，并在最终消息里去重展示。

失败降级

工具失败时，不应该把内部 JSON 或异常堆栈直接暴露给用户。ActMe 的策略是：

- 工具返回结构化错误。
- Agent 观察错误后尝试换工具、换来源或降级回答。
- 最终回复中只说明用户需要知道的失败原因。

例如：

- Bing HTML 失败，可以尝试 Bing Gecko 或其他搜索后端。
- GeckoView 读不到正文，可以尝试 HTTP 文本抽取。
- Python 报语法错误，可以修改代码后再次执行。
- Python 返回空回复且没有文件时，App 会要求 Agent 继续，不让用户手动补一句“继续”。
- ADB 连接失败，提示用户重新测试并保存连接端口。

与三驾马车的关系

Agentic Loop 是三驾马车的调度层：

- 内置浏览器负责信息获取。
- 内置 Python 负责确定性处理。
- 内置 ADB 负责本机观察和操作。

Loop 让它们可以组合成连续工作流，而不是孤立工具。

设计边界

当前实现是低配版 agentic workflow，不是完整任务队列系统：

- 没有跨进程任务恢复。
- 没有长期后台无人值守执行。
- 工具预算较保守。
- 高风险操作仍需要用户确认和可见步骤。

这个设计适合移动端个人 Agent：可见、可控、可暂停、可继续。

BUILTIN_ADB.MD

内置 ADB

内置 ADB 是 ActMe 的本机 Android 环境操作能力。它让 Agent 在用户授权后，可以通过无线调试连接本机 `adbd`，执行 `adb shell` 命令，用于观察系统状态、读取 UI 信息、查看日志和进行受控输入操作。

目标

内置 ADB 的目标不是替代完整桌面 adb，而是给移动端 Agent 一个轻量执行通道：

- 查看当前窗口、Activity、输入法、屏幕状态。
- 读取 logcat。
- 使用 `uiautomator dump` 获取 UI 层级。
- 执行 `input tap`、`input text`、`input keyevent` 等简单 GUI 操作。
- 配合多步 Agent 循环，实现“观察 -> 决策 -> 操作 -> 再观察”。

主要实现

核心代码位置：

```
app/src/main/java/com/actme/app/data/agent/AdbSkillEngine.kt
app/src/main/java/com/actme/app/data/agent/SystemSkillExecutor.kt
app/src/main/java/com/actme/app/ui/AdbOverlayService.kt
```

实现分层：

- `AdbSkillEngine`：负责保存 host/port、配对、测试连接、执行 shell。
- `SystemSkillExecutor`：接收 Agent 的 `adb_shell` system call。
- `AdbOverlayService`：系统悬浮窗，用于在无线调试设置页上方完成配对输入。

为什么需要悬浮窗

Android 无线调试配对时，系统会显示配对码弹窗。这个弹窗离开或关闭后，配对端口和验证码可能失效。

因此 ActMe 不能把配对表单放在普通 App 页面里。正确方式是：

- 用户打开 ActMe 的“内置 ADB”。
- ActMe 启动悬浮窗。
- ActMe 打开系统开发者选项。
- 用户进入无线调试并点击“使用配对码配对设备”。
- 用户保持系统配对码弹窗不关闭，在 ActMe 悬浮窗里输入配对端口和验证码。

部分系统还要求开启：

- 显示在其他应用上层。
- 允许在“设置”上重叠显示。

否则悬浮窗可能不能显示在系统设置页面上方。

悬浮窗交互

悬浮窗默认是小窗，避免挡住系统设置页面：

- 标题：内置 ADB。
- 按钮：设置、展开、关闭。
- 状态区：显示当前提示或错误。

点击“展开”后显示滚动表单：

- Host
- 配对端口
- 验证码
- 连接端口
- shell 测试命令

展开区域高度固定，内容通过滚动查看。

连接、配对或 shell 失败时：

- 状态区显示红字错误。
- 同时弹出 Toast。

Agent 调用示例

```
{
  "type": "adb_shell",
  "command": "dumpsys window | head -50",
  "timeout_ms": 15000
}
```

兼容别名：

- adb_shell
- adb
- run_adb

命令可以写成：

```
dumpsys window | head -50
```

也可以写成：

```
adb shell dumpsys window | head -50
```

执行器会自动去掉 adb shell 前缀。

连接持续性

点击“测试并保存连接”成功后，ActMe 会保存当前 `host:port`。

之后悬浮窗可以关闭，Agent 仍可继续调用 `adb_shell`。每次调用时，App 会读取保存的连接配置，并临时建立 ADB 连接执行命令。

需要注意：

- 无线调试连接端口可能在关闭/重开无线调试、重启手机后变化。
- 端口变化后需要重新测试并保存连接。
- 配对关系通常会保留，除非用户撤销配对、清除应用数据或系统重置无线调试状态。

安全边界

ADB 是高权限能力，必须谨慎使用：

- 默认优先执行只读命令，例如 `dumpsys`、`settings get`、`pm list`、`logcat -d`、`uiautomator dump`。
- 只有用户明确要求时才执行 GUI 操作，例如点击、输入、返回键。
- 不应擅自执行删除、卸载、清数据、改权限、改系统设置等破坏性命令。

建议 Agent 遵循：

- 先观察。
- 再解释准备执行什么。
- 对高风险命令等待用户确认。
- 执行后再次观察结果。

与其他能力配合

内置 ADB 可以和浏览器、Python 组合：

- ADB 获取当前 UI XML。
- Python 解析 XML，找出控件坐标。
- ADB 执行点击/输入。
- ADB 再次 dump UI 验证结果。

这使 ActMe 具备低配版 GUI 自动化能力。

BUILTIN_BROWSER.MD

内置浏览器

内置浏览器是 ActMe 的联网信息获取能力。它让 Agent 可以搜索网页、打开指定 URL，并读取渲染后的页面文本。

目标

内置浏览器解决两个问题：

- 大模型知识可能过期，必须联网确认。
- 普通 HTTP 抓取经常读不到动态页面，需要真实浏览器渲染。

因此 ActMe 使用 GeckoView 作为内置浏览器基础，并在 Agent 工具层提供：

- `web_search`
- `browse_url`
- `browser_url`
- `web_browse`
- `open_url`

其中 `browse_url` / `browser_url` 会打开网页并返回渲染后的文本。

主要实现

核心代码位置：

```
app/src/main/java/com/actme/app/data/agent/SystemSkillExecutor.kt
app/src/main/java/com/actme/app/data/agent/GeckoSearchEngine.kt
app/src/main/java/com/actme/app/ui/GeckoDebugActivity.kt
```

实现分层：

- `SystemSkillExecutor`：接收 Agent 的 system call，决定调用搜索还是网页浏览。
- `GeckoSearchEngine`：管理 GeckoView runtime/session，执行搜索或网页渲染读取。
- `GeckoDebugActivity`：设置页里的“内置浏览器”调试入口，用于人工验证页面加载和搜索效果。

Agent 调用示例

搜索：

```
{
  "type": "web_search",
  "query": "2026年5月31日国际金价实时价格"
}
```

打开网页：

```
{
  "type": "browse_url",
  "url": "https://www.boc.cn/fimarkets/"
}
```

搜索策略

当前搜索后端由 `SystemSkillExecutor` 管理。优先级包含：

- Bing Gecko
- Bing HTML
- DuckDuckGo API
- DuckDuckGo HTML
- Baidu
- SearXNG

实际使用中，ActMe 倾向于让 Agent 多搜索、多浏览来源进行确认，尤其是：

- 当前信息。
- 金融价格。
- 官网公告。
- 政策法规。
- 产品参数。
- 需要高准确性的事实。

页面读取

浏览网页时，App 会尽量返回渲染后的页面文本。Agent 可据此：

- 判断页面是否权威。
- 提取正文。
- 继续打开页面中的相关链接。
- 对多个来源做交叉验证。

如果搜索结果只提供面包屑式 URL，例如：

```
https://www.boc.cn > fimarkets
```

Agent 可以还原为：

```
https://www.boc.cn/fimarkets
```

再调用 `browse_url`。

UI 与调试

设置页提供“内置浏览器”入口，打开后可以：

- 手动输入 URL。
- 查看 GeckoView 页面加载。
- 验证搜索 URL。
- 查看 logcat 中的浏览器行为。

这个入口主要用于调试，不是最终用户主流程。

限制

- 某些页面需要登录或验证码，内置浏览器不一定能自动读取。
- 页面文本提取可能包含导航、广告、脚本生成文本。
- Bing 等搜索结果在不同网络、地区、Cookie 状态下可能变化。
- 动态页面即使渲染成功，也可能需要额外交互才能看到数据。

与其他能力配合

内置浏览器通常和 Python 配合使用：

- `web_search` 找来源。

- `browse_url` 读取页面。
- `python_exec` 清洗、排序、去重、计算。
- Agent 总结结论并引用来源文本。

BUILTIN_PYTHON.MD

内置 Python

内置 Python 是 ActMe 的确定性计算和文件处理能力。它让 Agent 不只依赖语言模型推理，而是可以运行代码完成计算、解析、表格处理、Excel 读写、通用文件生成和脚本复用。

目标

内置 Python 主要解决这些问题：

- 数学和数据计算需要确定性。
- 网页或文本提取后需要清洗、排序、去重、统计。
- Excel、PDF、CSV、图片、JSON、文本等文件需要读取、分析或生成。
- 复杂任务需要可复用脚本，而不是每次重新生成一段代码。

主要实现

核心代码位置：

```
app/src/main/java/com/actme/app/data/agent/PythonSkillEngine.kt
app/src/main/python/agent_python.py
app/src/main/java/com/actme/app/data/agent/SystemSkillExecutor.kt
```

实现分层：

- `PythonSkillEngine`：初始化 Chaquopy Python runtime，并从 Kotlin 调用 Python。
- `agent_python.py`：提供沙箱执行、输入输出、Excel 读写、通用文件跟踪、脚本保存和编译。
- `SystemSkillExecutor`：接收 Agent 的 `python_exec` system call，执行后把结果返回给多步循环。

Agent 调用示例

```
{
  "type": "python_exec",
  "code": "emit({'answer': 2 + 2})",
  "input": "",
  "timeout_ms": 3000
}
```

Python 代码里可以使用：

```
input_text
input_json
emit(value)
set_result(value)
result
workspace_dir
report_font_dir
read_excel(path, max_rows=200, max_sheets=10)
write_excel(filename, sheets)
write_report(markdown_text, base_name="report", title=None)
save_script(name, source)
load_script(name)
list_scripts()
compile_script(name)
run_script(name)
```

从 1.2.0 开始，Python import 策略从“允许列表”改为“默认允许已安装包和标准库，少数危险能力受限”。当前随 APK 打包的常用库包括 `numpy`、`pandas`、`openpyxl`、`Pillow`、`matplotlib`、`reportlab`、`markdown`、`fpdf2` 和 `fonttools`，因此表格分析、Excel、图片、Markdown 和 HTML 生成可以直接走 `python_exec`。PDF 报告优先走 Android 侧 `html_to_pdf`。

Excel 能力

ActMe 已内置 `openpyxl`、`pandas` 和 `numpy`，支持：

- 读取 `.xlsx`
- 读取 `.xls`
- 生成 `.xlsx`

- 从聊天中返回生成文件路径
- 从系统“打开方式/分享”接收 Excel 文件并载入当前会话

示例：

```
{
  "type": "python_exec",
  "code": "data = read_excel('/path/to/file.xlsx')\nemit(data)",
  "input": "",
  "timeout_ms": 10000
}
```

生成 Excel：

```
{
  "type": "python_exec",
  "code": "path = write_excel('result.xlsx', {'Sheet1': [['name', 'score'], ['A', 95]]})\nemit({'file': path})",
  "input": "",
  "timeout_ms": 10000
}
```

通用文件输出

Python 执行前后会扫描 `agent_workspace`。只要脚本在工作区内生成或修改文件，执行器会自动把文件加入 `output_files`。

Agent 也可以显式声明输出文件：

```
{
  "type": "python_exec",
  "code": "open('summary.txt', 'w', encoding='utf-8').write('done')",
  "output_files": ["summary.txt"]
}
```

Repository 会在 Agent loop 的每一轮工具执行后收集文件，并在最终聊天消息中去重显示。Chat UI 会为存在的文件显示打开按钮。

当前通用打开支持：

- Excel: `.xlsx`, `.xlsm`, `.xls`

- 文本和结构化数据: `.txt`, `.md`, `.csv`, `.json`
- 文档: `.pdf`
- 图片: `.png`, `.jpg`, `.jpeg`
- 其他文件回退到系统通用打开器

报告生成

从 1.3.0 开始, 报告类任务优先使用 `write_report`, 而不是让 Agent 手写 ReportLab 或 FPDF 样板代码:

```
markdown_text = """# 示例报告

## 摘要
这是中文报告内容。
"""

files = write_report(markdown_text, "reports/example_report", title="示例报告")
emit(files)
```

`write_report` 会:

- 生成 UTF-8 BOM 编码的 Markdown 文件, 方便 Windows 编辑器正确识别中文。
- 把 Markdown 转成 HTML。
- 返回 `markdown`、`html` 和 `output_files`, 执行器也会自动扫描工作区新增文件。

如果用户需要 PDF, Agent 应在 `write_report` 生成 HTML 后继续调用 `html_to_pdf`:

```
{
  "type": "html_to_pdf",
  "url": "reports/example_report.html",
  "output_files": ["reports/example_report.pdf"]
}
```

`html_to_pdf` 使用 Android WebView 渲染 HTML 并通过系统打印管线写出 PDF, 避免在 Python/Chaquopy 中维护复杂的 HTML-to-PDF native 依赖链。

`report_font_dir` 是 App 拷贝出来的字体目录, 可能位于 `agent_workspace` 外。它是运行时资源, 不是生成文件目录。Agent 不需要把字体路径写进 `output_files`。为了让文件按钮稳定显示, 模型应把报告、表格、图片等用户可见产物写入 `agent_workspace` 下的相对路径。

运行期脚本

Agent 可以保存脚本：

```
save_script("tools/analyze.py", source)
```

然后编译：

```
compile_script("tools/analyze.py")
```

再执行：

```
run_script("tools/analyze.py")
```

这让 Agent 可以逐步构建自己的轻量工具库。

安全边界

内置 Python 是沙箱式能力：

- 默认允许导入标准库和已安装包。
- Python 层不额外拦截文件写入、删除、重命名；实际能否成功交给 Android App 沙箱和系统权限决定。Prompt 仍要求 Agent 把用户可见的生成文件写到 `agent_workspace` 相对路径，便于 App 收集并显示打开按钮。
- 读取类操作尽量放开，以便第三方库读取自身资源、App 打包资源和用户可读文件。
- 禁止或限制包安装、进程控制、native code、系统 shell 等危险能力，例如 `pip`、`venv`、`subprocess`、`ctypes`、`multiprocessing`、`os.system`。
- 常见 cache/config 路径会指向工作区，例如 `HOME`、`MPLCONFIGDIR`、`XDG_CACHE_HOME`。

与其他能力配合

典型链路：

- 内置浏览器搜索和读取网页。
- Python 解析文本、清洗数据、计算结果。
- Python 清洗数据、计算结果或生成 Excel/PDF/CSV/图片等文件。

- Agent 把结论和文件按钮返回给用户。

SKILL_MEMORY_DESIGN.MD

Skill 与 Memory 设计

ActMe 的 Skill 与 Memory 是 Agent 个性化和长期可用性的基础。Memory 负责记录用户长期信息，Skill 负责沉淀可复用做事方法。

设计目标

Memory 与 Skill 分别解决不同问题：

- Memory：用户是谁、有什么偏好、目标、关系、健康状态、近期事项。
- Skill：遇到某类任务时，Agent 应该采用什么固定流程、检查表或行动模板。

二者共同让 ActMe 从“每轮重新理解用户”变成“持续适应用户”的个人 Agent。

数据位置

主要代码位置：

```
app/src/main/java/com/actme/app/data/local/Entities.kt
app/src/main/java/com/actme/app/data/local/Daos.kt
app/src/main/java/com/actme/app/skills/MemorySeeder.kt
app/src/main/java/com/actme/app/skills/SkillSeeder.kt
app/src/main/java/com/actme/app/data/agent/ActMeAgent.kt
app/src/main/java/com/actme/app/data/repo/ActMeRepository.kt
```

当前 Memory 和 Skill 都存储在本地 Room 数据库。

Memory 数据模型

Memory 使用 `MemoryItemEntity`：

```
data class MemoryItemEntity(  
    val id: Long,  
    val category: String,  
    val content: String,  
    val source: String,  
    val updatedAt: Long  
)
```

典型分类包括：

- 系统
- 短期目标
- 长期目标
- 个人焦虑
- 近期烦恼
- 个人偏好
- 人际关系
- 健康状况
- 学习工作

其中“系统”类 memory 用于 App 身份和固定能力说明，一般不允许 Agent 随意写入。

Memory 写入

Agent 可以在输出 JSON 中写入：

```
{  
  "memory_updates": [  
    {  
      "category": "长期目标",  
      "content": "用户希望长期提升英语口语能力。"  
    }  
  ]  
}
```

Repository 接收后写入本地数据库。

Memory 写入原则：

- 只记录长期有用的信息。

- 不把一次性闲聊全部写入。
- 不写入过于敏感或不确定的信息。
- 不重复记录已存在的明显相同内容。

Memory 注入

每轮对话前，ActMe 会把部分 memory 注入 system prompt:

- 系统 memory 优先注入。
- 用户 memory 取最近或最相关的一部分。
- 注入内容控制数量，避免 prompt 过长。

注入格式类似：

用户记忆：

- [长期目标] 用户希望长期提升英语口语能力。
- [个人偏好] 用户喜欢直接、可执行的建议。

Skill 数据模型

Skill 使用 `SkillEntity`：

```
data class SkillEntity(  
    val id: Long,  
    val name: String,  
    val description: String,  
    val triggerKeywords: String,  
    val actionTemplate: String,  
    val enabled: Boolean,  
    val createdAt: Long  
)
```

当前 Skill 是轻量结构：

- `name`：技能名。
- `description`：用途描述。
- `triggerKeywords`：触发关键词，JSON 字符串数组。
- `actionTemplate`：触发后追加给用户或 Agent 的行动模板。
- `enabled`：是否启用。

Skill 写入

Agent 可以输出：

```
{
  "skill_updates": [
    {
      "name": "考试复习计划",
      "description": "当用户提到考试和复习时，生成分阶段计划。",
      "trigger_keywords": ["考试", "复习", "备考"],
      "action_template": "先确认考试日期和科目，再拆分每日复习任务，并设置提醒。"
    }
  ]
}
```

Repository 将其保存为本地 Skill。

Skill 注入

每轮对话前，ActMe 会注入启用的 Skill 列表：

```
当前技能：
- 考试复习计划：["考试","复习","备考"]
```

此外，Repository 会根据用户输入匹配关键词，触发本地 skill hint，把 `actionTemplate` 追加到最终回复或过程上下文。

与 Claude/Codex Skill 的关系

当前 Skill 模型是轻量版，不等于 Claude/Codex 的目录型 skill。

Claude/Codex 风格 skill 通常包含：

- `SKILL.md`
- `scripts`
- `references`
- `templates`
- `assets`

ActMe 当前已有工具 `tools/adapt_claude_skills.py`，可以下载 Claude-style skill 并转换为当前 App 能识别的轻量 preload JSON，同时保留完整 skill 内容到 `assets`。

未来更完整的方案应该扩展 Skill schema:

- source
- content
- resourcePath
- version
- format
- enabled

并在 prompt 构造时按任务相关性注入完整 SKILL.md 或摘要。

Skill 与 Memory 的区别

Memory 记录“关于用户的事实”:

用户每周三晚上有英语课。
用户喜欢表格式计划。
用户近期在准备期末考试。

Skill 记录“做事方法”:

当用户要求备考计划时，先确认考试日期，再拆每日任务，再设置提醒。

简单判断:

- 这是用户个人信息 -> Memory。
- 这是可复用流程 -> Skill。

与 Agentic Loop 的关系

Memory 和 Skill 是 Agentic Loop 的上下文层:

- Memory 告诉 Agent 用户是谁、偏好什么、当前目标是什么。
- Skill 告诉 Agent 遇到某类任务时应该如何行动。
- Agentic Loop 根据这些上下文决定是否调用浏览器、Python、ADB 或日程能力。

学生场景示例

Memory:

[长期目标] 用户准备考研英语。

[学习工作] 用户每周一、三、五晚上复习数学。

[个人偏好] 用户希望计划以每日 `checklist` 展示。

Skill:

考试复习计划：先确认考试日期、科目、当前水平，再生成每日计划和提醒。

论文资料整理：先联网搜索多个来源，再按主题提炼观点和引用链接。

Excel 数据分析：先读取表格结构，再用 `python` 汇总统计，最后生成结果文件。

风险与边界

- Memory 不应过度记录敏感信息。
- Skill 不应沉淀错误流程。
- 用户应该能查看、删除和禁用 memory/skill。
- 长期应支持去重、合并、过期和可信来源标记。

Memory 和 Skill 的核心价值是让 Agent 更像“长期助手”，而不是一次性问答模型。

ActMe 创意设计文档：聚焦 Agentic Loop、内置工具链与端侧 GUI Agent 探索。